
BCA EXAMINATION NOTES

Course Code: CA-121

CA-121 Object Oriented Programming in C++

Exam Pattern:

Q1. Attempt Any Six – 6 marks

Q2. Attempt Any Three – 6 marks

Q3. Attempt Any Two – 6 marks

Q4A. Attempt Any One – 4 marks | Q4B. Compulsory – 2 marks

Q5. Attempt Any One – 6 marks

UNIT 1: OOPs in C++

1.1 OOP Features

Object

An object is a real-world entity that has a state (attributes) and behaviour (methods). In C++, an object is an instance of a class. Example: A car object has attributes (color, speed) and behaviours (accelerate, brake).

Class

A class is a user-defined blueprint or template from which objects are created. It defines data members (attributes) and member functions (methods). A class does not occupy memory; only its objects do.

Data Encapsulation

Encapsulation is the wrapping of data and methods into a single unit (class). It restricts direct access to data and allows access only through defined methods, protecting data integrity.

Data Abstraction

Abstraction means showing only essential features and hiding implementation details. In C++, abstract classes and interfaces achieve abstraction. Users interact with objects without knowing internal implementation.

Delegation

Delegation is the principle of handing off a task to another object. Instead of inheriting, an object delegates responsibility to another object, promoting code reuse through composition.

Inheritance

Inheritance allows a new class (derived) to acquire properties and behaviours of an existing class (base). Promotes code reuse. Types: Single, Multiple, Multilevel, Hierarchical, Hybrid.

Polymorphism

Polymorphism means 'many forms'. The same function/operator behaves differently based on context. Types: Compile-time (function overloading, operator overloading) and Runtime (virtual functions).

Message Communication

Objects communicate by sending messages (calling methods) to each other. A message specifies the method name and arguments. This is the mechanism of interaction in OOP.

1.2 Class Specification and Defining Members

Syntax of a Class:

```
class ClassName {  
private:  
    // data members  
public:  
    // member functions  
};
```

Member functions can be defined inside or outside the class. Outside definition uses scope resolution operator (::).

Example: `void Student::display() { cout << name; }`

1.3 Object and Access Specifiers

Object creation: `ClassName objectName;` or `ClassName obj = ClassName();`

Access Specifiers control visibility of class members:

- `private`: Accessible only within the class (default)
- `public`: Accessible from anywhere
- `protected`: Accessible within class and derived classes

1.4 Constructors and Destructors

Constructor

A constructor is a special member function automatically called when an object is created. It has the same name as the class and no return type. Used to initialize objects.

Types of Constructors

- **Default Constructor**: No parameters. Called automatically when object is created without arguments.
- **Parameterized Constructor**: Takes parameters. Used to initialize object with specific values.
- **Copy Constructor**: Takes a reference to an object of the same class. Creates a copy of an existing object.
- **Constructor Overloading**: Multiple constructors with different parameter lists in the same class.

Destructor

A destructor is a special member function called automatically when an object goes out of scope or is deleted. It has the same name as the class preceded by `~`. Used to release resources. Example: `~Student() { }`

1.5 Friend Class and Friend Function

Friend Function: A non-member function declared with keyword `friend` inside a class, having access to private and protected members.

Friend Class: When an entire class is declared as friend of another class, all its member functions can access private members of the other class.

Example: `class A { friend class B; private: int x; };` — class B can access A's private x.

UNIT 2: Inheritance

2.1 Types of Inheritance

- **Single Inheritance**: One base class, one derived class. `class B : public A { };`
- **Multiple Inheritance**: One derived class inherits from multiple base classes. `class C : public A, public B { };`
- **Multilevel Inheritance**: Chain of inheritance. `A -> B -> C`
- **Hierarchical Inheritance**: Multiple derived classes from one base class.
- **Hybrid Inheritance**: Combination of two or more types of inheritance.

2.2 Member Accessibility and Visibility Modes

Visibility modes define how base class members are inherited:

- public inheritance: public→public, protected→protected, private→not inherited
- protected inheritance: public→protected, protected→protected, private→not inherited
- private inheritance: public→private, protected→private, private→not inherited

2.3 Virtual Base Class

Virtual base class is used to solve the diamond problem in hybrid/multiple inheritance. When multiple derived classes inherit from the same base, the virtual keyword ensures only one copy of the base class is inherited.

Example: `class A {};` `class B : virtual public A {};` `class C : virtual public A {};` `class D : public B, public C {};`

Without virtual, D would contain two copies of A. With virtual, only one shared copy exists.

2.4 Benefits of Inheritance

- Code Reusability: Existing code can be reused in derived classes
- Extensibility: New features can be added without modifying base class
- Reduced Development Time: Less code to write
- Maintainability: Bug fixes in base class propagate to derived classes
- Real-world modelling: Models is-a relationships accurately

2.5 Virtual and Pure Virtual Functions

Virtual Function

A virtual function is a member function declared with keyword `virtual` in the base class and overridden in derived classes. Enables runtime polymorphism (late binding). The correct function is called based on the actual object type, not the pointer type.

Pure Virtual Function

A pure virtual function has no body in the base class and must be overridden in derived classes. Declared as: `virtual return_type funcName() = 0;`

2.6 Abstract Class

A class containing at least one pure virtual function is called an abstract class. It cannot be instantiated (no objects can be created from it). It serves as a base class providing a common interface. Derived classes must implement all pure virtual functions to become concrete classes.

UNIT 3: Polymorphism and Templates

3.1 Concept of Polymorphism

Polymorphism (Greek: 'many forms') allows a single interface to represent different underlying data types or a single function to work in different ways. It is one of the fundamental pillars of OOP.

3.2 Types of Polymorphism

- Compile-time (Static) Polymorphism: Resolved at compile time. Achieved by function overloading and operator overloading.
- Runtime (Dynamic) Polymorphism: Resolved at runtime. Achieved by virtual functions and function overriding.

3.3 Function Overloading

Function overloading allows multiple functions with the same name but different parameter lists (number, type, or order of parameters) in the same scope.

Example: `int add(int a, int b); float add(float a, float b); int add(int a, int b, int c);`

The correct function is selected at compile time based on the arguments. Return type alone cannot differentiate overloaded functions.

3.4 Operator Overloading

Operator overloading allows user-defined types (classes) to use operators like `+`, `-`, `*`, `==`, etc. by giving them special meaning for those types.

Unary Operator Overloading

Syntax: `return_type operator op() { ... }` — No parameter for member function version.

Example: `void operator++() { x++; }` — Overloads `++` for a class.

Binary Operator Overloading

Syntax: `return_type operator op(ClassName obj) { ... }` — One parameter for member function version.

Example: `Complex operator+(Complex c) { return Complex(real+c.real, imag+c.imag); }`

Using Friend Functions

When using friend functions for operator overloading, both operands are passed as arguments: `friend Complex operator+(Complex c1, Complex c2)`

Without Friend Functions

Using member functions, the left operand is the calling object and right is the argument: `Complex operator+(Complex c)` — called as `c1+c2` means `c1.operator+(c2)`

3.5 Class Template and Function Template

Function Template

A function template creates a family of functions for different data types using a single definition.

Syntax: `template <typename T> T max(T a, T b) { return (a>b)?a:b; }`

Class Template

A class template creates a family of classes for different data types.

Syntax: `template <class T> class Stack { T data[100]; int top; ... };`

The actual type is specified when creating objects: `Stack<int> s; Stack<float> fs;`

UNIT 4: Exception Handling and Stream Computation

4.1 Concept of Exception Handling

Exception handling is a mechanism to handle runtime errors gracefully without abrupt program termination. It separates error-handling code from normal code, making programs more readable and robust.

4.2 Types of Exceptions

- Synchronous Exceptions: Occur during normal program execution (e.g., division by zero, array out of bounds, file not found)

- Asynchronous Exceptions: Occur due to external events (hardware failure, signals) — not handled by C++ exception mechanism
- Standard Exceptions: Derived from `std::exception` (`bad_alloc`, `bad_cast`, `out_of_range`, `logic_error`, `runtime_error`)
- User-defined Exceptions: Custom exception classes created by programmer by inheriting from `std::exception`

4.3 Exception Handling Techniques

C++ uses three keywords: `try`, `catch`, `throw`

`try` block: Contains code that might throw an exception.

`throw`: Throws an exception when an error occurs. `throw` expression;

`catch` block: Catches and handles the exception. Can have multiple `catch` blocks for different types.

Syntax: `try { if(b==0) throw "Division by zero"; cout << a/b; } catch(const char* msg) { cout << "Error: " << msg; } catch(...) { cout << "Unknown error"; // catch-all }`

Multiple `catch` blocks handle different exception types. `catch(...)` is a catch-all handler.

4.4 Stream Computation with Console

C++ uses streams for input/output. Streams are defined in `<iostream>`.

- `cin` – Standard input stream (keyboard)
- `cout` – Standard output stream (monitor)
- `cerr` – Standard error stream (unbuffered)
- `clog` – Standard log stream (buffered)

Operators: `>>` (extraction – read from stream), `<<` (insertion – write to stream)

Example: `cout << "Enter: "; cin >> x;`

4.5 Stream Computation with Files

File I/O uses `<fstream>` library with three classes:

- `ifstream` – Input file stream (reading)
- `ofstream` – Output file stream (writing)
- `fstream` – Both input and output

Opening file: `fstream f; f.open("file.txt", ios::in | ios::out);`

Modes: `ios::in` (read), `ios::out` (write), `ios::app` (append), `ios::binary` (binary mode)

4.6 Read and Write Operations on File

Writing to file: `ofstream fout; fout.open("data.txt"); fout << "Hello"; fout.close();`

Reading from file: `ifstream fin; fin.open("data.txt"); fin >> str; fin.close();`

Functions: `getline(fin, str)` – reads full line; `get()` – reads single character; `put()` – writes single character; `eof()` – returns true at end of file; `seekg/seekp` – move file pointer.

DEFINITIONS (2-Mark Ready)

Class: A class is a user-defined blueprint or template for creating objects. It contains data members and member functions that define the properties and behaviours of objects.

Object: An object is an instance of a class. It has a state (data) and behaviour (methods) and represents a real-world entity.

Encapsulation: Encapsulation is the OOP principle of binding data and the methods that operate on it into a single unit (class), hiding internal details from outside access.

Inheritance: Inheritance is the mechanism by which a derived class acquires properties and behaviours of a base class, enabling code reuse and hierarchical classification.

Polymorphism: Polymorphism means the ability of a function or operator to act differently based on context, achieved through function overloading, operator overloading, and virtual functions.

Constructor: A constructor is a special member function with the same name as the class and no return type, automatically invoked when an object is created to initialize it.

Destructor: A destructor is a special member function invoked automatically when an object goes out of scope, used to free resources. It has the class name preceded by ~.

Virtual Function: A virtual function is a member function declared with the virtual keyword in the base class, enabling runtime polymorphism by allowing derived class override.

Abstract Class: An abstract class is a class that contains at least one pure virtual function. It cannot be instantiated and serves as a base class defining a common interface.

Exception: An exception is an abnormal condition or error that occurs during program execution and disrupts the normal flow, handled by try-catch-throw mechanism.

20 MOST IMPORTANT QUESTIONS WITH ANSWERS

Q1. What are the features of OOP? [2M]

Features of OOP:

1. Object: Real-world entity with state and behaviour
2. Class: Blueprint for creating objects
3. Encapsulation: Binding data and functions together
4. Abstraction: Hiding implementation details
5. Inheritance: Deriving new classes from existing ones
6. Polymorphism: Same name, different behaviour
7. Message Communication: Objects interact by calling methods

Q2. Define class and object with example. [4M]

CLASS: A class is a user-defined blueprint that defines data members and member functions.

Example:

```
class Student {  
private:  
    int rollNo;  
    char name[20];  
public:  
    void setData(int r, char n[]) { rollNo=r; strcpy(name,n); }  
    void display() { cout<<};  
}
```

OBJECT: An object is an instance of a class that occupies memory.

Creation: Student s1; s1.setData(101, "Ram"); s1.display();

Each object has its own copy of data members.

Q3. Explain constructors and their types. [6M]

A constructor is a special member function automatically called when an object is created. It has the same name as the class and no return type.

1. DEFAULT CONSTRUCTOR: No parameters.

```
Student() { rollno=0; name="None"; }
```

Called: `Student s; // automatically`

2. PARAMETERIZED CONSTRUCTOR: Takes arguments.

```
Student(int r, char n[]) { rollno=r; strcpy(name,n); }
```

Called: `Student s(101, "Ram");`

3. COPY CONSTRUCTOR: Creates a copy of existing object.

```
Student(Student &s;) { rollno=s.rollno; strcpy(name,s.name); }
```

Called: `Student s2(s1);`

4. CONSTRUCTOR OVERLOADING: Multiple constructors with different signatures.

```
Destructor: ~Student() { cout<<"Object destroyed"; }
```

Called automatically when object goes out of scope.

Q4. What is friend function? Write its advantages. [4M]

A friend function is a non-member function declared inside a class with the keyword friend. It has access to private and protected members of the class without being a member.

Syntax:

```
class MyClass {  
private:  
int x;  
public:  
friend void show(MyClass m);  
};  
void show(MyClass m) { cout<
```

Advantages:

1. Allows access to private data from outside the class
2. Useful for operator overloading (binary operators)
3. Enables cooperation between different classes

Disadvantages:

1. Breaks encapsulation principle
2. Increases coupling between classes

Q5. Explain types of inheritance with examples. [6M]

Inheritance allows a derived class to reuse code from a base class.

1. SINGLE INHERITANCE: One base, one derived.

```
class Animal { };
```

```
class Dog : public Animal { };
```

2. MULTIPLE INHERITANCE: Multiple base classes.


```
class Teacher { }; class Researcher { };  
class Professor : public Teacher, public Researcher { };
```

3. MULTILEVEL INHERITANCE: Chain of inheritance.

```
class A { }; class B : public A { }; class C : public B { };
```

4. HIERARCHICAL INHERITANCE: One base, many derived.

```
class Shape { };  
class Circle : public Shape { };  
class Square : public Shape { };
```

5. HYBRID INHERITANCE: Combination of above types.

May cause diamond problem; solved using virtual base class.

Q6. What is virtual base class? Why is it needed? [4M]

Virtual base class is used to prevent multiple copies of a base class in hybrid inheritance (diamond problem).

Problem (Diamond Problem):

```
class A { int x; };  
class B : public A { };  
class C : public A { };  
class D : public B, public C { }; // D has two copies of A!
```

Solution – Virtual Base Class:

```
class B : virtual public A { };  
class C : virtual public A { };  
class D : public B, public C { }; // Only one copy of A
```

virtual keyword ensures only one instance of A is created regardless of how many times it appears in the inheritance hierarchy. D's constructor initializes A directly.

Q7. Explain virtual functions and pure virtual functions. [6M]

VIRTUAL FUNCTION: A member function declared with virtual keyword in base class and overridden in derived class. Enables runtime polymorphism.

Example:

```
class Shape {  
public:  
    virtual void draw() { cout<<"Drawing Shape"; }  
};  
class Circle : public Shape {  
public:  
    void draw() { cout<<"Drawing Circle"; }  
};  
Shape *s = new Circle();  
s->draw(); // prints "Drawing Circle" – runtime polymorphism
```

PURE VIRTUAL FUNCTION: Declared with =0, no body in base class. Derived class must implement it.

```
virtual void draw() = 0; // pure virtual
```

ABSTRACT CLASS: Class with at least one pure virtual function.

Cannot create objects of abstract class. Forces derived classes to provide implementation.

Q8. What is polymorphism? Explain its types. [4M]

Polymorphism means 'many forms'. The same entity behaves differently in different situations.

1. COMPILE-TIME (STATIC) POLYMORPHISM:

Resolved at compile time. Achieved by:

- a) Function Overloading: Same function name, different parameters.
 - b) Operator Overloading: Same operator, different meaning for objects.
- Faster execution.

2. RUNTIME (DYNAMIC) POLYMORPHISM:

Resolved at runtime using virtual functions.

Base class pointer can point to derived class object.

Correct function called based on actual object type.

Slightly slower due to vtable lookup.

Example of runtime: `Shape *s; s = new Circle(); s->draw(); // calls Circle's draw()`

Q9. Explain function overloading with example. [4M]

Function overloading allows multiple functions with the same name but different parameter signatures in the same scope.

Rules:

- 1. Same function name
- 2. Different number, type, or order of parameters
- 3. Return type alone cannot differentiate

Example:

```
class Calculator {
public:
    int add(int a, int b) { return a+b; }
    float add(float a, float b) { return a+b; }
    int add(int a, int b, int c) { return a+b+c; }
};

Calculator c;
c.add(3,4); // calls int version
c.add(3.5f, 2.1f); // calls float version
c.add(1,2,3); // calls three-parameter version
```

Compiler selects appropriate version at compile time.

Q10. Explain operator overloading with an example of + operator. [6M]

Operator overloading allows user-defined classes to use C++ operators in a meaningful way.

Syntax: `return_type operator op(params) { ... }`

Example – Overloading + for Complex numbers:

```

class Complex {
private:
float real, imag;
public:
Complex(float r=0, float i=0) { real=r; imag=i; }
Complex operator+(Complex c) {
return Complex(real+c.real, imag+c.imag);
}
void display() { cout<<";
Complex c1(2,3), c2(4,5);
Complex c3 = c1+c2; // calls c1.operator+(c2)
c3.display(); // outputs: 6+8i

```

Rules:

1. Cannot overload: :: . * sizeof ?:
2. Cannot change precedence or number of operands
3. At least one operand must be user-defined type

Q11. What are templates? Explain function template and class template. [6M]

Templates allow writing generic code that works with any data type. C++ has two types:

FUNCTION TEMPLATE: Creates a generic function.

template

```

T maximum(T a, T b) {
return (a > b) ? a : b;
}

```

// Usage:

```
cout << maximum(3, 7); // int version
```

```
cout << maximum(3.5, 2.1); // float version
```

Compiler generates appropriate version automatically.

CLASS TEMPLATE: Creates a generic class.

template

```

class Stack {
T arr[100]; int top;
public:
void push(T val) { arr[++top] = val; }
T pop() { return arr[top--]; }
};

```

// Usage:

```
Stack intStack;
```

```
Stack strStack;
```

Each specialization generates a separate class.

Q12. Explain exception handling with try, catch, throw. [6M]

Exception handling manages runtime errors gracefully.

Keywords:

1. try: Encloses code that may cause exceptions

2. throw: Throws an exception when error occurs
3. catch: Catches and handles the exception

Syntax and Example:

```
try {  
    int a, b;  
    cin >> a >> b;  
    if(b == 0) throw "Division by Zero!";  
    cout << a/b;  
} catch(const char* msg) {  
    cout << "Error: " << msg;  
} catch(int e) {  
    cout << "Integer exception: " << e;  
} catch(...) {  
    cout << "Unknown exception"; // catch-all  
}
```

Flow: If throw executes, control transfers to matching catch. If no match, program terminates. catch(...) catches anything. Multiple catch blocks for different exception types.

Q13. What is file handling in C++? Explain read and write operations. [6M]

File handling allows storing and retrieving data from disk using file streams.

Header: #include

Classes:

1. ofstream – Output (write) file stream
2. ifstream – Input (read) file stream
3. fstream – Both read and write

Writing to File:

```
ofstream fout("data.txt");  
fout << "Student: Ram" << endl;  
fout << "Marks: 90" << endl;  
fout.close();
```

Reading from File:

```
ifstream fin("data.txt");  
string line;  
while(getline(fin, line)) {  
    cout << line << endl;  
}  
fin.close();
```

File Open Modes: ios::in, ios::out, ios::app, ios::ate, ios::binary

Functions: open(), close(), eof(), read(), write(), seekg(), seekp()

Q14. What is encapsulation and abstraction? [4M]

ENCAPSULATION: Encapsulation is the OOP feature of bundling data (variables) and methods (functions) that operate on the data into a single unit called a class. It also involves hiding internal data

from direct outside access (data hiding).

Example: In class Employee, salary is private; only getSalary() public method can access it.

Benefit: Protects data integrity, increases security, easy maintenance.

ABSTRACTION: Abstraction means hiding complex implementation and showing only necessary features to the user.

Example: When you call a function display(), you don't need to know how it works internally.

Achieved by: Abstract classes (with pure virtual functions), interfaces.

Benefit: Reduces complexity, allows focusing on what an object does rather than how.

Q15. Explain access specifiers in C++ with example. [4M]

Access specifiers control the visibility and accessibility of class members:

1. PRIVATE:

Members accessible only within the class.

Default access for class members.

Example: private: int salary; – cannot be accessed from outside.

2. PUBLIC:

Members accessible from anywhere (inside/outside class).

Example: public: void display(); – can be called from main().

3. PROTECTED:

Members accessible within the class and its derived classes.

Not accessible from outside.

Example: protected: int id; – derived class can access id.

Example:

```
class Person {  
private: int age;  
protected: string name;  
public:  
void setAge(int a) { age = a; }  
void show() { cout<<};
```

Q16. What is destructor? How is it different from constructor? [2M]

Destructor: A special member function called automatically when an object goes out of scope or is destroyed. It releases resources allocated by the object.

Syntax: ~ClassName() { }

Example: ~Student() { cout<<"Destroyed"; }

Differences:

Constructor: Called when object is created; can be overloaded; can have parameters.

Destructor: Called when object is destroyed; cannot be overloaded; no parameters; only one per class.

Q17. What is the difference between function overloading and function overriding? [4M]

FUNCTION OVERLOADING:

1. Multiple functions with same name but different parameters in same class/scope

2. Compile-time polymorphism

3. Differentiated by parameter list (number, type, order)

4. Occurs in same class

Example: `int add(int,int)` and `float add(float,float)`

FUNCTION OVERRIDING:

1. Derived class provides its own implementation of base class virtual function
2. Runtime polymorphism
3. Same name, same parameters
4. Occurs in different classes (base and derived)

Example: Base has virtual `void draw()`; Derived overrides `draw()`.

Q18. Explain the concept of abstract class. [2M]

An abstract class is a class that contains at least one pure virtual function. It cannot be instantiated (objects cannot be created directly from it). It serves as a base class providing a common interface for derived classes.

Syntax: `virtual void draw() = 0; // pure virtual`

class Shape is abstract if it has pure virtual `draw()`.

Derived classes MUST implement all pure virtual functions to become concrete (instantiable).

Q19. What is multiple inheritance? What is the diamond problem? [4M]

MULTIPLE INHERITANCE: A derived class inherits from two or more base classes.

`class C : public A, public B { }; // C inherits from both A and B`

DIAMOND PROBLEM: Occurs in hybrid inheritance when a class inherits from two classes that both inherit from the same base class.

`class A { int x; };`

`class B : public A { };`

`class C : public A { };`

`class D : public B, public C { }; // D gets two copies of A's x`

Problems: Ambiguity in accessing `x`; wasted memory for duplicate data.

Solution: Virtual base class.

`class B : virtual public A { };`

`class C : virtual public A { };`

Now D has only one shared copy of A.

Q20. What are the advantages of OOP over procedural programming? [2M]

Advantages of OOP over procedural:

1. Data Security: Encapsulation hides and protects data from unauthorized access.
2. Code Reuse: Inheritance allows reusing existing code in new classes.
3. Modularity: Programs are divided into objects, easier to manage.
4. Maintenance: Changes in one class don't affect others.
5. Real-world modelling: OOP models real-world entities naturally.
6. Polymorphism: Single interface for different types simplifies code.
7. Scalability: Easy to extend programs with new classes.